

Artificial Intelligence - Weekly Assessment-2

1. What is a list? Mention three important characteristics of lists.

A **list** in Python is a built-in, mutable, ordered sequence used to store a collection of items in a single variable.

Three Important Characteristics:

1. **Ordered:** Elements maintain the exact position in which they are inserted.
2. **Mutable:** You can modify, add, or remove elements after the list is created without changing its memory identity.
3. **Allows Duplicates & Heterogeneous Types:** A single list can contain duplicate values and mix different data types (e.g., integers, strings, floats, booleans).

2. What is the difference between append and extend?

Both methods add elements to an existing list, but they handle iterable inputs differently:

- **.append(element):** Adds its argument as a **single element** to the end of the list, regardless of its data type (if you pass a list, it adds the whole list as a single sublist).
- **.extend(iterable):** Iterates over its argument and adds **each element individually** to the end of the list.

```
# append example
a = [1, 2]
a.append([3, 4])
print(a) # Output: [1, 2, [3, 4]]

# extend example
b = [1, 2]
b.extend([3, 4])
print(b) # Output: [1, 2, 3, 4]
```

3. Write a program to find the largest and smallest values in a list.

```
numbers = [42, 12, 89, 7, 63, 25]

# Using built-in functions
smallest = min(numbers)
largest = max(numbers)

print("Smallest value:", smallest)
```

```
print("Largest value:", largest)
```

4. Write a program to calculate the sum and average of numbers stored in a list.

```
numbers = [10, 20, 30, 40, 50]

total_sum = sum(numbers)
average = total_sum / len(numbers) if len(numbers) > 0 else 0

print("Sum:", total_sum)
print("Average:", average)
```

5. Write a program to remove duplicate values from a list.

```
numbers = [1, 2, 2, 3, 4, 4, 4, 5, 1]

# Method 1: Using set (Order not guaranteed)
unique_numbers = list(set(numbers))
print("Unique values (set method):", unique_numbers)

# Method 2: Preserving original order
unique_ordered = list(dict.fromkeys(numbers))
print("Unique values (ordered method):", unique_ordered)
```

6. What is a tuple? How is it different from a list?

A **tuple** is an immutable, ordered sequence of items enclosed in parentheses ().

Feature	List	Tuple
Syntax	Square brackets []	Parentheses ()
Mutability	Mutable (can be modified)	Immutable (cannot be modified)

Performance	Slightly slower, uses more memory	Faster, memory-efficient
Use Case	Collections that change frequently	Fixed data structures (e.g., coordinates)

7. What is tuple unpacking? Explain with a real-life example.

Tuple unpacking allows you to assign elements of a tuple directly into individual variables in a single operation.

Real-Life Example: Extracting GPS coordinates (Latitude, Longitude) or a point's Cartesian coordinates (x, y) .

```
# GPS Coordinate tuple
location = (12.9716, 77.5946)

# Unpacking into individual variables
latitude, longitude = location

print("Latitude:", latitude)    # Output: 12.9716
print("Longitude:", longitude)  # Output: 77.5946
```

8. What is a set? Why are sets useful for removing duplicates?

A **set** is an unordered collection of unique elements enclosed in curly braces `{}`.

Why sets are useful for removing duplicates:

1. **Guaranteed Uniqueness:** Sets automatically discard duplicate values upon creation.
2. **High Efficiency:** Searching and hash table-based duplicate elimination in sets runs in $O(1)$ average time complexity, making it significantly faster than iterating through a list.

9. Write a program to find common elements between two collections.

```
collection1 = [1, 2, 3, 4, 5]
collection2 = [3, 4, 5, 6, 7]

# Convert to sets and find intersection
```

```
common_elements = list(set(collection1) & set(collection2))

print("Common elements:", common_elements)
```

10. What is a dictionary? Explain keys and values.

A **dictionary** is an unordered (ordered by insertion since Python 3.7), mutable mapping structure stored as key-value pairs (**{key: value}**).

- **Key:** A unique identifier used to look up data. Keys must be **immutable** data types (such as strings, numbers, or tuples).
- **Value:** The data associated with a key. Values can be of **any data type** and can contain duplicate values across different keys.

```
student = {
    "name": "Alice", # "name" is key, "Alice" is value
    "age": 21,      # "age" is key, 21 is value
    "grade": "A"    # "grade" is key, "A" is value
}

print(student["name"]) # Accessing value using its key
```